Data Structures Assignment - 2

D. Sandesh Goyal
CDS/0925/2025
Section - 2

1.) Reverse a string using stack

```
#include <stdio.h>
#include <string.h>

char stack[100];
int top = -1;

void push(char ch) {
    stack[++top] = ch;
}

char pop() {
    return stack[top--];
}

int main() {
    char str[100];
    printf("Enter string: ");
    scanf("%s", str);

    int len = strlen(str);

    for (int i=0; i<len; i++)
        push(str[i]);

    printf("Reversed string: ");
    for (int i=0; i<len; i++)
        printf("%c", pop());

    return 0;
}
```

Output :-

Input :- DATA
output :- ATAD

2.) Balanced Parentheses

```
#include <stdio.h>
#include <string.h>

char stack[100];
int top = -1;

void push(char ch) {stack[++top] = ch;}
void pop() {top--;}

int main() {
    char exp[100];
    printf("Enter expression: ");
    scanf("%s", exp);

    for(int i=0; i<strlen(exp); i++) {
        if(exp[i] == '(')
            push(exp[i]);

        else if(exp[i] == ')') {
            if(top == -1) {
                printf("Not Balanced");
                return 0;
            }
            pop();
        }
    }
    if(top == -1) {
        printf("Balanced Expression");
    }
    else {
        printf("Not Balanced");
    }
    return 0;
}
```

3) Next Greater Element

```
#include <stdio.h>
int main() {
    int arr[] = {4,5,16,9,56};
    int n = 5;
    for (int i=0; i<n; i++) {
        int found = 0;
        for (int j = i+1; j<n; j++) {
            if (arr[j] > arr[i]) {
                printf("%d -> %d\n", arr[i], arr[j]);
                found = 1;
                break;
            }
        }
        if (!found)
            printf("%d -> -1\n", arr[i]);
    }
    return 0;
}
```

4) Printer Queue Simulation (using queue)

```
#include <stdio.h>
#include <string.h>

#define MAX 5

char queue[MAX][50];
int front = -1, rear = -1;

void enqueue(char doc[]) {
    if (rear == MAX - 1) {
        printf("Queue is Full!\n");
        return;
    }
    if (front == -1) front = 0;
    rear++;
    strcpy(queue[rear], doc);
    printf("Document added to queue\n");
}

void dequeue() {
    if (front == -1 || front > rear) {
        printf("No documents to print\n");
        return;
    }
    printf("Printing: %s\n", queue[front]);
    front++;
}

void display() {
    if (front == -1 || front > rear) {
        printf("Queue is Empty\n");
        return;
```

```
    printf("Pending Documents : \n");
    for (int i = front; i <= rear; i++) {
        printf("%s \n", queue[i]);
    }
}
int main() {
    int choice;
    char doc[50];
    while (1) {
        printf("\n 1. Add Document \n 2. Print Document \n 3. Display
                queue \n 4. Exit \n");
        printf("Enter choice: ");
        scanf("%d", &choice);
        getchar();
        switch (choice) {
            case 1:
                printf("Enter document name: ");
                fgets(doc, 50, stdin);
                doc[strcspn(doc, "\n")] = 0;
                enqueue(doc);
                break;
            case 2:
                dequeue();
                break;
            case 3:
                display();
                break;
            case 4:
                return 0;
```

```
        default :
            printf (" Invalid choice \n");
    }
  }
}
```

5.) Circular Queue (Menu Driven)

```
#include <stdio.h>
#define SIZE 5

int cq[SIZE];
int front = -1, rear = -1;

void enqueue (int value) {
    if ((rear+1) % SIZE == front) {
        printf (" Queue is Full \n");
        return;
    }
    if (front == -1) front = 0;
    rear = (rear + 1) % SIZE;
    cq[rear] = value;

    printf (" Inserted : %d \n", value);
}
void dequeue ( ) {
    if (front == -1) {
        printf (" Queue is Empty \n");
        return;
    }
    printf (" Removed : %d \n", cq[front]);
```

```
    if (front == rear) {
        front = rear = -1;
    }
    else {
        front = (front + 1) % SIZE;
    }
}

void peek() {
    if (front == -1) {
        printf("Queue is Empty \n");
        return;
    }
    printf("Front element : %d \n", cq[front]);
}

void display() {
    if (front == -1) {
        printf("Queue is Empty \n");
        return;
    }
    printf("Queue elements : \n");
    int i = front;
    while (1) {
        printf("%d", cq[i]);
        if (i == rear) break;
        i = (i+1) % SIZE;
    }
    printf("\n");
}
```

```
int main() {
    int choice, value;

    while (1) {
        printf ("\n1. Enqueue \n2. Dequeue \n3. Peek \n4.
                \n5. Exit \n");
        printf ("Enter choice : ");
        scanf ("%d", &choice);

        switch (choice) {
            case 1:
                printf ("Enter value : ");
                scanf ("%d", &value);
                enqueue (value);
                break;

            case 2:
                dequeue();
                break;

            case 3:
                peek();
                break;

            case 4:
                display();
                break;

            case 5:
                return 0;

            default:
                printf ("Invalid choice \n");

        }
    }
```